

Sleep phase 안에서 실제로 무엇을 계산하는가

어려운 Study paper 를 그림과 예로 다시 읽는 안내서

읽는 법 이 문서는 원 Study 의 결론을 바꾸지 않고, 질문→세 문장→직관→예/반례→그림→대안→미증명 지점 순서로 풀어 쓴다. 파란색 training 용어는 별도 Background paper 의 해당 개념으로 연결된다. 주장 ID 와 figure ID 는 원 Study 와 동일하다.

Contents

E05 · Sleep phase 안에서 실제로 무엇을계산하는가	2
이 권이 답할질문	2
먼저 세 문장	2
직관	2
예와 반례	3
그림 읽기	4
대안과 비교	5
아직 모르는것	5
더 깊이 읽기	6
문서 경계와 사용법	7

E05 · Sleep phase 안에서 실제로 무엇을 계산하는가

이 권이 답할질문

“자는 동안 학습한다” 를 실제 연산 단계로 풀면 무엇이 남는가? Sleep phase 는 하나의 algorithm 이 아니라 data construction, selection, replay, distillation, adapter optimization, evaluation, state publication 을 엮은 pipeline 이다. 어느 단계를 쓰느냐에 따라 compute 와 memory traffic 이 완전히 달라진다.

이 권은 sleep mechanism taxonomy 를 따라 **STC-C015 · synthesis · confidence high**, **STC-C019 · author_claim · confidence high**, **STC-C023 · synthesis · confidence high**, **STC-C024 · synthesis · confidence high**, **STC-C035 · author_claim · confidence high** 을 연산 관점으로 설명한다.

먼저 세 문장

1. External-memory sleep 의 기본 loop 는 **episode 수집→추출→중복제거→충돌 해결→summary/graph/index 갱신→검증**이다. Gradient 가 없어도 generation 과 search 가 크다.
2. Parametric sleep 은 **replay data 구성→teacher/student 또는 task loss 계산→backward→optimizer→evaluation→adapter/weight version 배포**다. LoRA 로 weight write 를 줄여도 activation 과 data movement 는 남는다.
3. Dreaming 이나 synthetic augmentation 은 data 부족을 줄이지만, 자기 output 을 다시 학습하는 recursive loop 이므로 real evidence anchor 와 quality gate 가 필요하다.

직관

Mechanism A — Replay

과거 data 또는 그것을 흉내 낸 sample 을 새 data 와 섞어 학습한다. 원 sample 을 보존하면 fidelity 가 높지만 storage·privacy cost 가 있다. 합성데이터로 replay 하면 저장량을 줄일 수 있으나 rare case 와 failure mode 를 generator 가 놓칠 수 있다. Replay 는 “기억을 다시 보여 주는 data plane” 이다.

Mechanism B — Distillation

Teacher 의 output distribution 이나 behavior 를 student state 로 옮긴다. 지식증류는 raw label 보다 richer signal 을 줄 수 있고, large context/fast memory 를 compact adapter 로 옮기는 데 쓰인다. 하지만 compression 은 무엇을 버릴지 결정한다. Teacher 가 틀렸거나 student capacity 가 부족하면 오류도 함께 압축된다.

Mechanism C — Data augmentation 과 dreaming

Model 이 스스로 candidate experience 를 만들고 evaluator 가 useful sample 을 고른다. 데이터 증강의 장점은 실제 log 에 없던 corner case 를 탐색할 수 있다는 점이다. Reward 나 downstream improvement 로 sample 을 고르면 보상설계가 data distribution 을 지배한다. “그럴듯함” 과 “학습 후 utility” 는 다르다.

Mechanism D — Adapter update

LoRA 는 full weight 대신 low-rank delta 를 학습한다. Trainable parameter 와 optimizer state 가 줄고 version 단위가 작아진다. 그러나 forward activation, backward gradient, base weight read 는 여전히 필요하다. 또 user 별 adapter 가 많아지면 batch 가 갈라지고 routing metadata 가 커진다.

Mechanism E — Consolidation 과 compaction

여러 fast memory 를 slow memory 로 옮기거나, 여러 episode 를 summary/graph 로 압축한다. 공고화는중요한 pattern 을 장기 state 로 promotion 하는 과정이다. External version 은 merge·supersede·index rebuild 이고, parametric version 은 distillation·fine-tuning·expert expansion 이다.

Mechanism F — RL-style optimization

Dream candidate 나 adapter policy 가 future task score 를 높이면 reward 를준다. 정책기울기를 쓸 수도 있고 best-of-N selection 처럼 gradient 없는 최적화를 쓸수도 있다. 핵심은 reward evaluation 이 sleep cost 의 일부이며, reward hacking 과 distribution shift 를 막아야 한다는 점이다.

시스템 모델링 관점

Sleep job 의 FLOP 만 계산하면 불완전하다. Episode read bytes, vector/graph random write, teacher rollout token, activation checkpoint, optimizer state, adapter snapshot, validation inference, failed version 폐기까지포함해야 한다.

예와 반례

External synthesis 한 cycle

1. 지난 24 시간의 consented episode 를 immutable log 에서 읽는다.
2. 후보 fact·preference·strategy 를 structured record 로 추출한다.
3. 기존 memory 와 semantic·temporal conflict 를 찾는다.
4. admit/merge/supersede/reject 를 결정하고 provenance edge 를 남긴다.
5. vector index 와 temporal graph 를 incremental update 한다.
6. held-out query 로 retrieval precision 과 contradiction 을 검사한다.
7. 통과한 generation 만 active pointer 로 전환한다.

이 cycle 은 foundation training 보다 가볍지만 embedding, LLM extraction, graph traversal, random write 가 섞인 heterogeneous workload 다.

Parametric adapter 한 cycle

1. External system-of-record 에서 high-reuse-low-conflict record 를선택한다.
2. Positive example, counterexample, deletion test, old-capability replay 를 dataset 으로 만든다.
3. Teacher output 과 ground truth 를 섞어 task/distillation loss 를계산한다.
4. Adapter 만 backward·optimizer update 한다.

두 개의 독립 축: update timing × memory medium

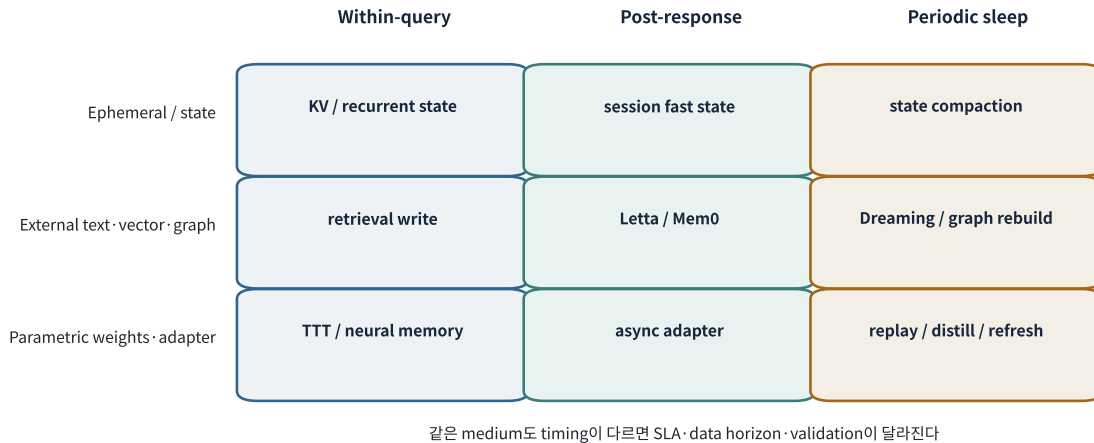


Figure 1: STC-F002. Update timing by memory-medium taxonomy. 원 Study 의저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를따른다.

5. New fact, old skill, locality, privacy, adversarial set 을 평가한다.
6. Threshold 를 통과하면 versioned adapter 를 publish 하고 canary traffic 에붙인다.
7. Regression 이면 pointer 를 이전 generation 으로 되돌린다.

반례 — Training loss 가 낮으니 consolidation 성공

Train loss 는 dataset 을 얼마나 잘 맞췄는지만 말한다. Old capability 가사라졌는지, unrelated answer 가 변했는지, delete request 가 반영되는지, future query latency 가 줄었는지는 별도 평가다. Sleep 의 output 은 model artifact 이므로 release gate 가 필요하다.

반례 — Dream sample 수를 늘리면 계속좋아짐

Synthetic sample 이 같은 generator bias 를 반복하면 quantity 가 diversity 를보장하지 않는다. Real data anchor 를 잃으면 distribution tail 이 줄어드는 model collapse 가 가능하다. Generated replay 는 free data 가 아니라 provenance 가 약한 derived data 다.

그림 읽기

Mechanism 을 memory medium 위에 배치해 보라. Replay·distillation 은 adapter 와 weight 쪽에, dedup·graph merge 는 external memory 쪽에, recurrent update 는 learned state 쪽에 놓인다. 동일 sleep scheduler 가서로 다른 kernel 과 storage pattern 을 부른다. 그래서 “sleep cluster” 는단일 GPU cluster 라기보다 GPU training pool, CPU/near-data preprocessing, graph/index tier 를 연결한 fabric 이다.

Failure-mode 그림은 pipeline 의 어느 지점에 control 을 넣어야 하는지 보여준다. Ingest 에는 consent-poison filtering, transform 에는 provenance·quality score, train 에는 replay·regularization, publish 에는 regression·canary, serve 에는 observability, delete 에는 lineage-driven rebuild 가 필요하다. Control 이 없는 sleep 은 오류를 durable 하게 만드는자동화다.

Failure mode마다 lifecycle control을 붙인다

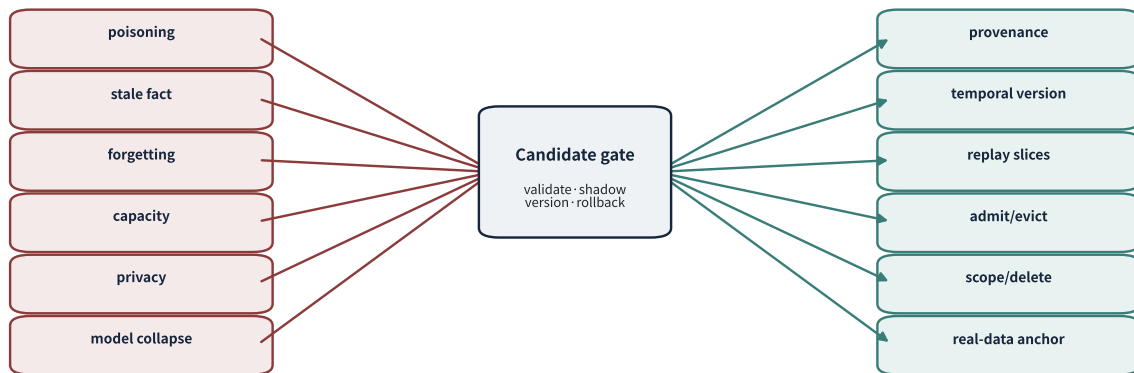


Figure 2: STC-F013. Failure modes and lifecycle controls. 원 Study 의 저자작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를따른다.

대안과 비교

mechanism	주된 연산	주된 state traffic	장점	핵심 risk
summary/d-edup graph update	generation, similarity extraction, traversal	episode read, record write random	해석 · 삭제 쉬움 시간 · 관계표현	lossy merge write amplification
replay SFT	forward/back- ward	activation, optimizer	강한 retention baseline	storage/privacy
distillation	teacher+stu- dent forward, backward	logits/activa- tion	compact behavior transfer	teacher error
LoRA	base forward+delta backward	base read, small weight write	작은 artifact	adapter explosion
dreaming/RL	rollout, reward, update	generated data, checkpoints	data exploration	recursive bias

Mechanism 선택은 “얼마나 AI 다운가” 가 아니라 memory utility 와 system cost 의 곱으로 결정해야 한다. Stable factual state 는 graph/record 가, repeated behavior 는 adapter 가, rare raw evidence 는 cold episode store 가 더 적합할 수 있다.

아직 모르는것

- Sleep cycle 별 end-to-end energy·wall-clock·data-movement breakdown 이 공개된 연구가 드물다.
- Teacher rollout, reward model, LoRA SFT 중 어느 향이 실제 utility 를 만드는지 ablation 이 충분하지 않다.
- Generated replay 가 rare tail 을 얼마나 보존하는지 장기 sequence 에서 측정되지 않았다.
- Multi-rate neural memory update 가 prefix cache reuse 와 함께 실제 serving throughput 을 높이는지 실증이 부족하다.

- Online consolidation 과 scheduled sleep 사이에서 optimal split 을 학습하는 scheduler 가 없다.

핵심

Sleep 은 “한 번 더 fine-tuning” 이 아니다. Evidence 를 data 로 만들고, state 를 갱신하고, 검증해 배포하는 작은 ML lifecycle 전체다.

더 깊이 읽기

- 원 분석: replay·distill·RL·external compaction mechanism
- Training Background: 증류, 데이터증강, 합성데이터, LoRA, 보상, 정책기울기, 공고화
- Claim route: **STC-C015** · synthesis · confidence high · **STC-C019** · author_claim · confidence high · **STC-C023** · synthesis · confidence high · **STC-C024** · synthesis · confidence high · **STC-C035** · author_claim · confidence high

문서 경계와 사용법

이 문서는 쉬운 설명본이지 새로운 증거원이 아니다. 수치 · 인용 · claim status 는 원 Study paper 와 claim map 이 우선한다. 학습 개념은 Training Background paper 를 함께 읽고, 논문 재현이나 외부 배포 판단에는 고정 source 와 rights audit 를 사용한다.